Knowledge Distillation and Deployment of a Command Parser on a Local Cloud

Cloud Computing (CS3081) Mini Project

Sai Veena Borra | Roll No. 230103032 | https://github.com/veenaborra/cloud-mini-project

Setup

I did the whole project on my Windows laptop (12 logical CPUs, no GPU). I worked inside WSL2 with Ubuntu 24.04 and used Docker Desktop, k3d v5.9.0 and k3s v1.30.6. The cluster has one server node and one agent node. The teacher, phi3.5, ran in Ollama for Windows. For training I used PyTorch 2.5.1 (CPU build) and Transformers 4.46.3.

In this report, when I call a number **measured**, I got it from an actual run and the file is in `results/`. Everything else, like the setup, my design choices and my guesses about why something happened, is my own reasoning and not a measurement.

Overall design. Ollama runs on the laptop itself. Inside the k3d cluster I used three namespaces. `data` has SeaweedFS, which gives S3-style object storage, with two buckets called `datasets` and `models`. `pipeline` is where the two labelling workers ran. `app` has my FastAPI service, with a Service, a Traefik Ingress and an autoscaler in front of it. Port 8080 on my laptop goes to Traefik and port 8333 goes to SeaweedFS. I did not put the model inside the Docker image. The service downloads it from `s3://models/<version>/` every time it starts. GitHub Actions runs on every push and tests the model that is stored as a GitHub Release.

Task 1: Object storage and parallel teacher labelling

The cluster is created from `cluster/k3d.yaml`. I pinned the k3s image version so it comes up the same way every time. SeaweedFS runs as one `weed server -s3` Pod with a 5 GiB volume. The S3 username and password are random keys made by `scripts/make_secrets.sh`. They are stored in a Kubernetes Secret and in a local `.s3.env` file that git ignores, and nowhere else. A small Python script creates the two buckets and uploads `commands_pool.jsonl` (197,541 bytes, measured).

Writing the prompt. I used phi3.5 with temperature 0, seed 42, and Ollama's JSON output option. When I first tried a simple prompt, the teacher wrote `"time": "07:00"` instead of "seven am", made up a place ("home"), and put the whole answer inside a markdown code block. So in `prompts/teacher_prompt.txt` I listed all eight intents with a short meaning for each, wrote the three slot rules clearly, and added nine examples that I wrote myself. None of them are from the validation set. I tested the prompt on a few commands from the pool and saw that it called a reminder an alarm, so I added a line saying "remind me ..." is always `calendar_set`. After that I did not change the prompt again.

How I split the work (sharding). I ran the workers as a single Indexed Job in `pipeline` with 2 completions running in parallel. Kubernetes gives each Pod an index, 0 or 1, and I pass that to the worker as `SHARD_INDEX`. `SHARD_COUNT` is 2. A worker picks line i of the pool if i mod 2 is equal to its own index, so worker 0 gets the even lines and worker 1 gets the odd lines. Every line number has only one remainder, so every command goes to exactly one worker. Nothing is missed and nothing is done twice, and each worker gets 1,500 commands even if similar commands are grouped together in the file. Each worker only writes to its own file in storage (`datasets/teacher/shard-<i>.jsonl`), so they never write to the same object. Since I knew the run would take hours, I made each worker save its progress every 25 commands and skip already saved ids when it restarts. The weak point is that both workers send requests to the same Ollama running on the same CPU, so the speed is limited by the teacher and not by how many workers I run.

What happened (measured). Both Pods ran together with index 0 and 1. The Job ended as Complete 2/2 with no restarts. The full run took about 5 hours 50 minutes, which is roughly 0.10 to 0.12 commands per second for each worker. A few times the workers could not resolve `host.k3d.internal` for a moment, but the retries in my worker handled it. When I checked coverage, all 3,000 ids were there, with none missing and none repeated.

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) invented slot	Duplicates	Kept
3,000	0	2	2	183	48	2,765

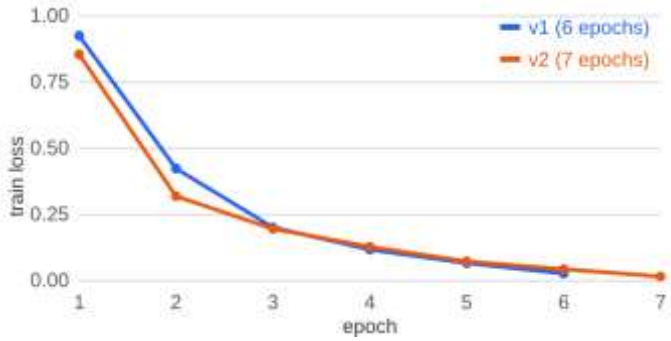
The numbers add up to the input: 0 + 2 + 2 + 183 + 48 + 2,765 = 3,000. My `clean.py` applies rules (a) to (d) first. After that it removes duplicates from the remaining records and keeps the one with the smallest id. The pool actually has 50 repeated commands, but in two cases one of the copies had already been removed by an earlier rule, so only 48 are counted as duplicates. The checksum in `train.sha256` is `8f7ade4bb69db11c02dda047269be0f2ed7110fe636b32695fb2e39d21a2da0e` and `sha256sum -c` passes. I got zero invalid JSON because of Ollama's JSON mode. Without it, the code-block answers I saw in my first test would have been rejected under rule (a).

I also went through the 187 rejected replies. The teacher sometimes rewrote things in its own words ("my weekly plan" became "this week"), corrected spelling ("twelveth" became "twelfth"), made up values like "office" or "unknown", and in one case copied "boston" from my prompt examples into an answer where it did not belong. I also realised rule (d) has a gap. It only checks that the value is somewhere in the command. It does not check that the value is in the correct slot, so a real phrase put in the wrong field still passes.

Limitations and improvements. SeaweedFS is a single Pod with one volume and no replication, so losing that volume would lose the data. Rule (d) cannot catch a correct phrase put in the wrong slot. Since both workers share one Ollama, adding workers barely helped. Next time I would use replicated SeaweedFS volumes, add a simple slot-type check (date words in date, places in place), and give each worker its own teacher instance so sharding actually speeds things up.

Task 2: Training the student

All the training settings are in `training/config.yaml`: seed 42, learning rate 5e-4 with a short warm-up and linear decay, batch size 16, 6 epochs for v1 and 7 for v2, maximum length 96 tokens, and the library versions. `make train` only reads this file. `make train SUBSET=50` trains on just the first 50 records, and on my laptop it finished in 55 seconds. Every training example is `Command: <text>`, then `JSON: <teacher reply>`, then an end-of-sequence token. I set the labels of the prompt part to -100, so the loss only counts the JSON and the end token. The model is not trained to repeat the command, only to write the answer and stop. I set the seed in the code for Python, NumPy and PyTorch, and the data is shuffled with a seeded generator, so running the same command gives the same model.



Training loss per epoch (measured, from `training/loss.csv` and `loss_v2.csv`). v1 took about 52 minutes and v2 about 48 minutes on CPU. The two lines are different even in epoch 1 because the learning rate schedule depends on the total number of steps.

Model (100 validation commands)	JSON valid	Intent	Date	Time	Place	Full frame
Teacher phi3.5 (3.8B), same prompt and settings	100	89	78	89	84	49
Pythia-70M, untrained	0	0	0	0	0	0
Student v1 (6 epochs), deployed	100	86	76	89	80	41
Student v2 (7 epochs)	100	83	75	86	78	40

What changed. The untrained Pythia model has no idea about this task. It just continues writing text, so none of its outputs were valid JSON. After fine-tuning on the teacher's answers, all outputs were valid JSON and the

student got 41% full frame compared to the teacher's 49%. So a model about 54 times smaller got around 84% of the teacher's score. Date was the weakest slot for both models. This makes sense to me because the student copies the teacher's way of answering, including the cases where the teacher and the SLURP labels disagree. For example, in some gold examples "weekend" is labelled as a time and not a date. I expected v2 to be better because its final training loss was lower (0.016 compared to 0.028), but it was not better on validation. I think this is a small amount of overfitting. Also, with only 100 validation commands, one point means one command, so I treat v1 and v2 as basically the same and I deployed v1.

Saving the model. `training/package.py` creates an archive of the model with a fixed file order, fixed timestamps and fixed owners, so packing the same files always gives the same bytes. It uploads the model files to `models/v1/` and `models/v2/` and writes `model.sha256`. I checked (measured) that the Release asset `model-v1.tar.gz` (235,010,542 bytes), the `.sha256` file next to it and `models/v1/model.sha256` all have the same hash: `ca581b47a7e866f3de4895d593aa4212c204924404c96b990f93ffc93d2e2c63`. When I packed it again on a fresh clone, I got the same hash.

Is Pythia-70M a good choice? For this small and fixed task, I think it is good enough. It is small enough to train and run on a CPU, it has an Apache-2.0 licence, and it is an English model like our data. But it is only a base model, it is not instruction tuned and it knows very little, so it cannot do the task at all without fine-tuning. It also cannot notice when the teacher gives a wrong answer.

Sequence-level vs logit-based distillation

(a) **What sequence-level distillation trains on:** the text that the teacher generates. The student is trained with normal cross-entropy loss to produce the same output the teacher wrote for each input. This is what I did, following Kim and Rush (2016).

(b) **What logit-based distillation trains on:** the teacher's full probability distribution over the vocabulary at each token position. It is usually done with a KL-divergence loss on softened logits. This way the student also learns which other tokens the teacher thought were likely, not just the final answer.

(c) **Why I could not use it here:** phi3.5 and Pythia have different tokenizers and different vocabularies (about 32k and 50k tokens). Their probability distributions are over different sets of tokens and the positions do not match, so they cannot be compared directly. Also, Ollama only returns the generated text and not the full logits.

(d) **One limitation of what I did:** the student can only learn what the teacher wrote. If a wrong answer passes my cleaning, for example "in two hours" saved as a date or "gym" saved as a place, the student learns it as correct. The student also never finds out how confident or unsure the teacher was.

Limitations and improvements. The student copies the teacher's mistakes, v2 overfitted a little, and 100 validation commands is a small sample, so small differences are just noise. I would keep part of `train.jsonl` aside to stop training when the score stops improving, and ask the teacher two or three times per command and keep the majority answer to get cleaner labels.

Task 3: The inference service in a container

The service is a FastAPI app that runs on port 8000. When it starts, a background thread downloads all the files under `MODEL_URI` from `S3_ENDPOINT`, loads the model, does one warm-up generation, and only then sets itself as ready. Before that, `/readyz` returns 503. `/healthz` returns 200 as long as the process is running. `/parse` generates up to `MAX_NEW_TOKENS` tokens. If `FAULT_INJECT=1`, it replaces the output with "not json". Then it checks the output against `schema.json`. If it is valid, it returns only the four keys with 200. If not, it returns `{"error": "invalid_model_output"}` with 500. The responses include `X-Model-Version` and `X-Served-By`, which is the Pod name. The S3 keys come only from the `s3-credentials` Secret. Each Pod has one CPU, so I let it run only one generation at a time.

The image is based on `python:3.11.10-slim`. I pinned all 44 dependencies, including indirect ones, used the CPU-only build of PyTorch, and ran the app as a non-root user. I added a `.dockerignore` so the `build/` folder (where my model files are) and the secrets file can never get copied into the image. It builds with one command: `docker build -f service/Dockerfile -t cmd-parser:1.0 .`

Check (container alone, --cpus 1 --memory 1g)	What I measured (results/limits_test.txt)
Image size	1.6 GB on disk, 340 MB compressed
Start-up	Model download 8.8 s (8 files), ready after 11.3 s; /readyz changed from 503 to 200
50 requests, one after another	All 50 returned 200; median 456 ms, slowest 587 ms
Peak memory (cgroup memory.peak)	649,838,592 bytes, about 620 MiB; 342.5 MiB after the test; not OOM-killed
FAULT_INJECT=1	HTTP 500 with exactly {"error":"invalid_model_output"}, headers still there

The peak memory is almost two times the normal memory. I think this is because while the model is loading, the file and the model tensors are both in memory at the same time. From this I understood that the memory limit should be set for the start-up peak and not for normal running.

Limitations and improvements. Each Pod runs one generation at a time, so under load requests wait in a queue. The 1 Gi memory limit is mostly unused after start-up, and the image is still 1.6 GB because of PyTorch. I would try int8 quantisation or ONNX Runtime to make each request faster and the image smaller, batch requests together, and lower the memory limit once the start-up peak is smaller.

Task 4: Kubernetes deployment, scaling and reliability

In `k8s/app/app.yaml` I have a Deployment with 2 replicas, CPU and memory requests of 500m and 512 Mi, and limits of 1 CPU and 1 Gi. The readiness probe checks `/readyz` every 2 seconds and the liveness probe checks `/healthz`. For rolling updates I used `maxSurge 1` and `maxUnavailable 0`, so a new Pod is started before an old one is removed. I also added a `preStop` sleep of 8 seconds, so a Pod keeps answering while Traefik stops sending traffic to it. The same file has the Service, the Ingress for `/parse`, and an HPA that scales between 1 and 2 replicas at 70% of the CPU request. I set the scale-down window to 60 seconds so I did not have to wait the default 5 minutes during testing.

Experiment	Result (measured)	File in results/
Endpoints while loading	For about 20 s both Pods showed 0/1 Running and the Service had no endpoints. After that both IPs appeared	readiness_evidence.txt
Autoscaling	Under load the CPU went to 199% of the request and the HPA scaled from 1 to 2 (throughput went from about 1.8 to 3.5 req/s, no failures). About 2 minutes after the load stopped it went back to 1	hpa.txt
Rolling update and rollback	1,234 requests, 0 failed. v1 served 923 and v2 served 312. After kubectl rollout undo the header showed v1 again	rolling_update.txt
Recovery	Deleted 1 of 2 ready Pods. It was back to 2/2 in about 11 s; 662 requests, 0 failed	recovery.txt
Cold start, 3 runs	Run 1: 12 and 10 s, run 2: 10 and 10 s, run 3: 11 and 10 s. Medians of each run are 11, 10 and 10.5 s, so 10.5 s. The logs show the model download (3.8 to 4.6 s)	cold_start.txt
Load balancing	Exactly two ready replicas with the standard command: 151 and 151 successful requests	loadtest_headers.json

My first recovery test failed. When I ran it the first time, the HPA had already reduced the Deployment to one replica. So when I deleted a Pod, I deleted the only ready one. The Service had no endpoints left, Traefik removed the route, and every request got a 404. These errors came back in about 3 ms, and Locust sends the next request immediately, so even a few seconds without a ready Pod gave 1,852 failed requests out of 2,448 (76%, measured). I set the HPA minimum to 2 and ran the test again, and that is the result in the table. I kept both runs in `results/`, because the failed one actually taught me the most. Recovery without an outage only works if another ready copy is already running.

Limitations of my local cluster. Both nodes are just containers on the same laptop, so this setup does not show fault tolerance across machines. If my laptop, Docker or WSL stops, everything stops. The HPA needs about a

minute of data from metrics-server before it can work, and metrics are collected only about every 15 seconds, so scaling is always a bit behind the load. Also, my Windows clock was 5.5 hours off in the beginning, which made Pod ages and some times wrong until I fixed it. All the timings for Tasks 3, 4 and 7 were taken after I synced the clock. Finally, with a minimum of one replica, the HPA leaves a single point of failure whenever there is no load.

Improvements. I would keep `minReplicas` at 2 and add a PodDisruptionBudget so one Pod is always ready, and run the same setup on a real multi-node cluster to actually test failure of a machine.

Task 5: Subnetting and network access control

I started from 10.20.0.0 and allocated from the largest subnet to the smallest. For each one I chose the smallest prefix where 2^(32 - prefix) - 2 is at least the number of hosts needed. My working is in `network/subnets.md`, and I checked the final table with Python's `ipaddress` module.

Subnet	Needed	Network	Prefix	Usable range	Broadcast	Usable
app	1,000	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0	/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Direction	Rule	What it does (network/nacl.json, public subnet)
Inbound	50	Deny everything from 198.51.100.0/24
Inbound	100, 110	Allow TCP 80 and TCP 443 from anywhere
Inbound	120	Allow TCP 22 only from the management subnet 10.20.5.128/27
Inbound	130	Allow TCP 1024-65535 from the app subnet 10.20.0.0/22 (replies coming back)
Outbound	100	Allow TCP 8000 to the app subnet
Outbound	110	Allow TCP 1024-65535 to anywhere (replies to clients)

`nacl_sim.py --check` gets all 6 sample packets right (measured, `results/nacl_check.txt`).

Why replies need separate rules. A NACL is stateless. It does not remember that it allowed a connection, so when the reply comes, it checks it again like a completely new packet. For example, a client connects to port 443, but the reply goes back to the random port the client used, somewhere between 1024 and 65535. If there is no outbound rule for that range, the default deny blocks the reply and the page never loads. Sample S05 checks exactly this. It is the same for replies coming back from the app subnet to the load balancer, which is why I added inbound rule 130.

Why rule order matters for 198.51.100.0/24. The rules are checked from the lowest number upwards and the first rule that matches decides. If my deny rule had a higher number than the allow rule for port 443, a packet from 198.51.100.9 to port 443 would match the allow rule first and get through. Because the deny is rule 50, which is lower than 100 and 110, it blocks sample S04. I also noticed a side effect. Since the reply rules have to allow the whole 1024 to 65535 range, an outbound packet to port 8000 on some other network would also be allowed. A stateless filter cannot tell the difference between a reply and a new connection.

Limitations and improvements. Because a NACL is stateless, the reply rules have to open the whole ephemeral range, which also lets some unwanted traffic through. The plan also leaves most of the /16 unused. In a real setup I would add stateful security groups on the instances alongside the NACL, and leave gaps between subnets so each one can grow later.

Task 6: Reproducibility, testing and CI/CD

From a fresh clone, the whole system starts with three commands: `git clone`, `cd` and `make up`. `make up` creates the Python environment and the cluster, generates new random S3 keys and puts them in Secrets, deploys SeaweedFS, creates the buckets and uploads the committed teacher shards and training set, builds both images

and imports them into k3d, downloads `model-v1` and `model-v2` from GitHub Releases, checks their SHA-256, uploads them to object storage, deploys the service and sends one test request. I actually tested this. I ran `make down`, cloned the repo into an empty folder, ran `make up`, and it ended with HTTP 200 and a correct answer (`results/make_up.log`). I added the step that uploads the teacher shards and training set a little after this test. `make down` deletes the cluster.

The workflow in `.github/workflows/ci.yml` runs in this order: lint with ruff, 7 unit tests with pytest, an evaluation job that downloads the model from the GitHub Release, checks its checksum and runs `score.py` on `validation.jsonl`, and then a quality gate with `--min-validity 95`. The tests check the cleaning rules, duplicate handling and the count total, the subnet table, the NACL sample packets, a normal `/parse` call with its headers, the 500 response for bad output and for `FAULT_INJECT`, and both probes. A gitleaks job scans the whole git history at the same time and finds no leaks.

The normal run with model-v1 passes: github.com/veenaborra/cloud-mini-project/actions/runs/41841008027. To show that the gate works, I ran it once with the untrained Pythia-70M. It failed at the gate with `QUALITY GATE FAILED: json_validity 0.0 < 95.0`: github.com/veenaborra/cloud-mini-project/actions/runs/41841071333. `AI_USE.md` explains where I used AI and how I checked the results.

Limitations and improvements. `make up` is slow on a fresh machine because it builds the images and downloads both models, and the CI evaluation only uses the 100 validation commands. All the k8s experiments were run by hand with scripts, not in CI. I would cache images in a registry, add a CI job that deploys to a temporary k3d cluster and runs a short smoke test, and pin the GitHub Actions versions by commit.

Task 7: Performance, cost and review

I measured all of these with the standard load-test command (4 users for 2 minutes through the Ingress) with exactly two ready replicas (`results/loadtest_*`, `results/top_pods.txt`).

p50	p95	Throughput	Error rate	CPU per Pod	Memory per Pod	Cold start
1,600 ms	1,900 ms	2.53 req/s (302 requests)	0% (0 failed)	about 998m, at the limit	391 to 406 Mi	10.5 s median

One request on its own takes about 456 ms. Under load, each Pod still handles only one request at a time, so with four users and two Pods, each request waits for about one other request to finish. I think that is why the median is around 1.6 s. Throughput was not the same in every run. During the rolling update it was 3.4 req/s, and I think the difference came from other things running on my laptop.

Cost per 1,000 requests, using the rate card (INR 3.50 per vCPU-hour and INR 0.45 per GiB-hour, billed on the limits):

1. One replica per hour = 1 vCPU x 3.50 + 1 GiB x 0.45 = **INR 3.95**
2. Two replicas per hour = 2 x 3.95 = **INR 7.90**
3. Requests per hour = 2.5334 req/s x 3,600 = **9,120.1**
4. Cost per 1,000 requests = 7.90 / 9,120.1 x 1,000 = **INR 0.866**

SLO. My target is that 99% of `/parse` requests return HTTP 200 within 30 seconds, measured over one standard load-test run of 2 minutes. 1% of 302 requests gives an error budget of **3 failed requests**. I used **0** of it. In comparison, my failed single-replica recovery test would have gone far beyond this budget.

Area	Evidence (measured)	Limitation I found	What I would improve
Data and storage	2,765 of 3,000 replies kept (92.2%); 183 removed as invented slots; labelling took about 5 h 50 min	Only one SeaweedFS Pod with no replication; rule (d) misses real text in the wrong slot; one shared teacher limits what sharding can do	Replicated SeaweedFS volumes; check slot types (date words vs places); one Ollama per worker

Area	Evidence (measured)	Limitation I found	What I would improve
Model quality	Full frame 41% vs teacher 49%; date 76%; 100% valid JSON	Copies the teacher's mistakes; v2 overfits a little; only 100 validation examples	Keep part of the training data for early stopping; ask the teacher a few times and take the majority answer
Latency and throughput	p50 1,600 ms, p95 1,900 ms, 2.53 req/s; 456 ms for one request	Only one generation at a time per Pod, so requests wait in a queue	Batch requests; int8 quantisation or ONNX Runtime; lower MAX_NEW_TOKENS since the JSON is under 30 tokens
Resource use	CPU about 998m per Pod (at the limit); memory about 400 Mi normally, 620 MiB peak at start-up	CPU is the bottleneck; about 60% of the 1 Gi memory limit is unused after start-up	Lower the memory limit to about 768 Mi; quantise the model to use less CPU per request
Reliability and recovery	Rolling update 0 of 1,234 failed; recovery 0 of 662 failed, back to 2/2 in about 11 s; cold start 10.5 s	With one replica, one deletion caused 76% failures; one laptop means no real node redundancy	Keep minReplicas at 2 and add a PodDisruptionBudget; use a real multi-node cluster
Security and cost	gitleaks found nothing; INR 0.866 per 1,000 requests	All namespaces share one S3 admin login; cost depends on a full CPU limit per replica	Separate S3 users with only the access they need (read-only for the app); smaller CPU limit after quantisation

References

Kim and Rush, Sequence-Level Knowledge Distillation, EMNLP 2016. Hinton, Vinyals and Dean, Distilling the Knowledge in a Neural Network, 2015. Bastianelli et al., SLURP, EMNLP 2020. EleutherAI, Pythia-70M model card. Kubernetes documentation on Deployments, probes, HPA, Secrets and rolling updates. k3d and K3s documentation. SeaweedFS S3 API documentation. AWS VPC guide on network ACLs. Google SRE Book, Service Level Objectives. Locust documentation.